Week 1 Lecture Notes: Overview and Course Launch

Hong Qin, Old Dominion University

Fall 2026

Contents

1 Topic 1: Overview of Generative Modeling and Why It Matters **1**

2 Topic 2: Main Model Families Covered in the Course **3**

3 Topic 3: Course Workflow, Submissions, Communication, and Project Expectations **5**

4 Topic 4: Introductory Computing Setup for Local or Cloud Execution **7**

How to read these notes. Week 1 is a light week. Topics 1 and 2 give the single probabilistic idea the course keeps returning to, plus one defining equation per model family. Topics 3 and 4 are operational: how the course runs and where your code will run. Each equation is followed by a plain-English reading.

1 Topic 1: Overview of Generative Modeling and Why It Matters

1.1 What a generative model is

Suppose we observe a dataset of N examples

$$x^{(1)}, x^{(2)}, \ldots, x^{(N)} \overset{\text{i.i.d.}}{\sim} p_{\text{data}}(x),$$

where each $x^{(i)}$ might be an image, a sentence, a molecule, or a sensor trace, and p_{data} is the unknown distribution that produced them. A **generative model** is a parameterized probability distribution $p_\theta(x)$ that we try to fit so that

$$p_\theta(x) \approx p_{\text{data}}(x).$$

In plain English: we do not just want to label data; we want to learn the process that could have produced it. If we succeed, three things become possible: we can *sample* new examples, we can *score* how plausible a given example is, and the model's internal variables often turn out to be a useful *representation* of the data.

Foster (*Generative Deep Learning*, Chapter 1) states the framework as two rules. **Rule 1:** samples from p_θ should look like draws from p_{data}. **Rule 2:** samples should be suitably different from the training examples themselves; a model that only memorizes its training set has not learned the distribution.

1.2 Density estimation versus sampling

The phrase "learn p_{data}" hides two distinct capabilities, and different model families trade them off differently.

Density estimation. Given a point x, return the number $p_\theta(x)$ (or $\log p_\theta(x)$). This is what you need for anomaly detection, compression, and model comparison. Autoregressive models and normalizing flows give it exactly; VAEs give a lower bound; GANs do not give it at all.

Sampling. Draw a fresh $x \sim p_\theta$. This is what you need for image synthesis, text generation, and simulation. GANs and diffusion models are excellent samplers; autoregressive models sample one dimension at a time; VAEs sample by first drawing a latent code.

Why the distinction matters: a model can be good at one and poor at the other. A GAN can produce photorealistic faces yet cannot tell you the probability of any particular face. Keeping the two goals separate makes the later comparison table in Topic 2 easy to read.

1.3 Likelihood and the maximum-likelihood objective

The classical way to fit p_θ is **maximum likelihood**: choose the parameters that make the observed data as probable as possible,

$$\theta^\star \;=\; \arg\max_\theta \; \mathbb{E}_{x\sim p_{\text{data}}}[\log p_\theta(x)] \;\approx\; \arg\max_\theta \; \frac{1}{N}\sum_{i=1}^{N} \log p_\theta\Big(x^{(i)}\Big)\,. \tag{1}$$

Reading the equation: the log turns a product of probabilities over N independent examples into a sum, which is numerically stable and easy to differentiate. The expectation over p_{data} is what we would ideally compute; the sample average on the right is what we can actually compute.

Maximizing likelihood is the same as minimizing a KL divergence to the data distribution:

$$\mathrm{KL}(p_{\text{data}} \,\|\, p_\theta) \;=\; \mathbb{E}_{x\sim p_{\text{data}}}[\log p_{\text{data}}(x)] \;-\; \mathbb{E}_{x\sim p_{\text{data}}}[\log p_\theta(x)]\,. \tag{2}$$

The first term does not depend on θ, so minimizing the KL over θ is exactly maximizing (1).
In plain English: the KL divergence is the average extra "surprise" you pay for describing data with the wrong distribution. Maximum likelihood chooses the θ that minimizes that surprise. When $\log p_\theta(x)$ cannot be computed exactly (VAEs, EBMs) or is not defined by the model at all (GANs), each family substitutes a different surrogate objective; that substitution is what distinguishes the families in Topic 2.

1.4 Discriminative versus generative modeling

A **discriminative** model learns the conditional distribution of a label given an input,

$$p_\phi(y \,|\, x), \qquad \text{for example } p(\text{“cat”} \,|\, \text{image}),$$

while a **generative** model learns the distribution of the input itself,

$$p_\theta(x), \qquad \text{or jointly } p_\theta(x, y) = p_\theta(x \,|\, y)\, p(y).$$

The contrast in one sentence: a discriminative model draws a boundary between classes; a generative model describes what each class looks like. The generative task is harder (the space of images is far larger than the set of labels) but richer: a model of $p(x, y)$ can classify, via Bayes' rule $p(y \,|\, x) \propto p(x \,|\, y)\, p(y)$, and can also generate, impute missing values, and detect out-of-distribution inputs. A discriminative model can only classify.

1.5 Why generative AI matters

Generative models now design candidate drug molecules and proteins, draft and debug code, synthesize images, video, and audio, power research assistants, and simulate environments for science and engineering. They also raise serious questions about training-data provenance, evaluation, hallucination, bias, energy use, and misuse. Understanding how they are formulated and trained (Modules 2 through 8) is the foundation both for using them well and for judging their limits.

Readings. Foster, *Generative Deep Learning*, Chapter 1; Goodfellow, Bengio, and Courville, *Deep Learning*, Chapter 20 (free online at https://www.deeplearningbook.org). Further overviews: Bishop and Bishop, *Deep Learning: Foundations and Concepts* (2024); Murphy, *Probabilistic Machine Learning: Advanced Topics* (2023).

2 Topic 2: Main Model Families Covered in the Course

Every family below is a different answer to one question: *how should we parameterize and train* $p_\theta(x)$*?* For each family these notes give the defining equation, a plain-English reading, and the module where it is treated. Table 1 summarizes the comparison.

2.1 Variational autoencoders (Module 2)

Introduce a latent variable z with a simple prior $p(z)$, typically $\mathcal{N}(0, I)$, and define the data density by integrating it out:

$$p_\theta(x) \;=\; \int p_\theta(x \mid z)\, p(z)\, dz. \tag{3}$$

The integral is intractable, so training uses an approximate posterior $q_\phi(z \mid x)$ (the encoder) and maximizes the evidence lower bound (ELBO):

$$\log p_\theta(x) \;\geq\; \underbrace{\mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction}} \;-\; \underbrace{\mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z))}_{\text{regularization}}. \tag{4}$$

In plain English: encode x to a distribution over codes z, decode z back to x, and reward faithful reconstruction while penalizing codes that stray from the prior. The penalty is what keeps the latent space smooth enough to sample from. Primary paper: Kingma and Welling (2014).

2.2 Generative adversarial networks (Module 3)

A generator G_θ maps noise $z \sim p_z$ to a sample $G_\theta(z)$; a discriminator D_ϕ outputs the probability that its input is real. They play a two-player minimax game:

$$\min_\theta \max_\phi \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D_\phi(x)] \;+\; \mathbb{E}_{z \sim p_z}[\log(1 - D_\phi(G_\theta(z)))]\,. \tag{5}$$

In plain English: the discriminator tries to tell real from fake; the generator tries to fool it. There is no explicit $p_\theta(x)$, so a GAN cannot evaluate likelihoods, but at the game's equilibrium the generator's samples match the data distribution. Training instability and mode collapse motivate the Wasserstein variants covered in Module 3. Primary paper: Goodfellow et al. (2014).

2.3 Autoregressive models (Module 4)

Order the dimensions of $x = (x_1, \ldots, x_n)$ and factor the joint density with the chain rule of probability:

$$p_\theta(x) \;=\; \prod_{i=1}^{n} p_\theta(x_i \,|\, x_{<i}), \qquad x_{<i} = (x_1, \ldots, x_{i-1}). \tag{6}$$

In plain English: predict the next pixel, token, or note given everything before it. Each conditional is a neural network (an RNN, an LSTM, or a transformer), the likelihood is exact and cheap to compute, and sampling proceeds one dimension at a time. GPT-style language models follow exactly this template at scale.

2.4 Diffusion models (Module 5)

Fix a forward Markov chain that gradually corrupts data into Gaussian noise over T steps, and learn the reverse chain that removes the noise:

$$q(x_t \,|\, x_{t-1}) \;=\; \mathcal{N}\Big(x_t;\; \sqrt{1-\beta_t}\, x_{t-1},\; \beta_t I\Big), \qquad p_\theta(x_{t-1} \,|\, x_t) \;=\; \mathcal{N}(x_{t-1};\; \mu_\theta(x_t, t),\; \Sigma_\theta(x_t, t)). \tag{7}$$

In plain English: destroying an image with noise is easy; the model learns to run that process backwards. Training reduces to a simple denoising regression at random noise levels, which is why diffusion trains far more stably than a GAN. Primary papers: Ho, Jain, and Abbeel (2020); Song et al. (2021).

2.5 Transformers (Module 6)

The transformer is an architecture rather than a probabilistic family. Its core operation is scaled dot-product attention over query, key, and value matrices:

$$\text{Attention}(Q, K, V) \;=\; \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{8}$$

In plain English: each position in a sequence looks at every other position, decides how relevant each one is (the softmax weights), and takes a weighted average of their values. Stacked with feed-forward layers and positional encodings, this replaces recurrence and is now the backbone inside most autoregressive language models, many diffusion models, and nearly all multimodal systems. Primary paper: Vaswani et al. (2017).

2.6 World models (Module 7)

A world model learns a compact predictive model of an environment so that an agent can plan or train inside its own imagined rollouts. In the Ha and Schmidhuber (2018) design, a VAE encodes each observation o_t to a latent z_t, and a recurrent mixture-density network predicts the next latent given the current latent, action, and hidden state:

$$p_\theta(z_{t+1} \,|\, z_t, a_t, h_t) \;=\; \sum_{k=1}^{K} \pi_k(h_t)\, \mathcal{N}\Big(z_{t+1};\; \mu_k(h_t),\; \sigma_k^2(h_t) I\Big). \tag{9}$$

In plain English: compress what the agent sees, learn to predict what it will see next, and let a small controller act on those predictions. This is the module that couples generative modeling most directly with reinforcement learning.

2.7 Multimodal models (Module 8)

Multimodal systems align representations across modalities, most often text and images. CLIP (Radford et al., 2021) trains an image encoder f and a text encoder g so that matching pairs score higher than all mismatched pairs in a batch of B:

$$\mathcal{L} \;=\; -\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(\langle f(x_i), g(t_i)\rangle/\tau)}{\sum_{j=1}^{B}\exp(\langle f(x_i), g(t_j)\rangle/\tau)}, \tag{10}$$

with τ a learned temperature (the symmetric text-to-image term is added in practice).
In plain English: pull each image toward its own caption and push it away from every other caption in the batch. The resulting shared embedding space is what text-conditioned image generators and vision-language assistants build on.

2.8 Two families covered only briefly

Normalizing flows use an invertible map $x = f_\theta(z)$ so that the change-of-variables formula gives an exact likelihood,

$$p_\theta(x) \;=\; p(z)\left|\det\left(\frac{\partial f_\theta^{-1}(x)}{\partial x}\right)\right|.$$

Energy-based models specify an unnormalized density

$$p_\theta(x) \;=\; \frac{\exp(-E_\theta(x))}{Z(\theta)}, \qquad Z(\theta) = \int \exp(-E_\theta(x))\,dx,$$

whose partition function $Z(\theta)$ is intractable; training uses contrastive divergence or score matching. EBMs are the conceptual bridge to score-based diffusion.

Family	Latent z?	Likelihood	Trained by	Sampling
VAE (M2)	yes	lower bound (ELBO)	ELBO maximization	one decoder pass
GAN (M3)	yes	not defined	minimax game	one generator pass
Autoregressive (M4)	no	exact	maximum likelihood	sequential, per dimension
Diffusion (M5)	yes	lower bound	denoising regression	iterative, many steps
Transformer (M6)	n/a	inherits from family	inherits from family	inherits from family
World model (M7)	yes	lower bound / MDN	VAE, MDN-RNN, controller	rollout in latent space
Multimodal (M8)	varies	contrastive	contrastive alignment	via attached decoder

Table 1: Comparison of the model families in course order. "Inherits" means the transformer takes on the properties of whichever family it is used inside (autoregressive LLM, diffusion backbone, or CLIP encoder).

Readings. Foster, Chapter 1 in full, then the first two or three pages of the chapters on VAEs, GANs, sequence models, world models, and transformers. Primary papers (all on arXiv): Kingma and Welling 1312.6114; Goodfellow et al. 1406.2661; Dinh et al. 1605.08803; Ha and Schmidhuber 1803.10122; Vaswani et al. 1706.03762; Ho et al. 2006.11239; Song et al. 2011.13456; Radford et al. 2103.00020. Companion code for Foster's 2nd edition: https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition.

3 Topic 3: Course Workflow, Submissions, Communication, and Project Expectations

This topic is the "how the course actually runs" reference. Keep it handy for the whole term.

3.1 Course workflow

The course is eight weeks long, fully asynchronous, and delivered through the university learning management system. All modules are open from the start of the term and stay open through the end; the posted weekly windows are the recommended cadence, not hard locks. Late work still carries a five percent deduction per day, so on-time work matters. The recommended rhythm each week is: read the module overview and the four topic pages; watch the recorded lecture; do the hands-on part of the technical assignment; post the technical assignment; post the project-track update; reply to at least one classmate. Plan for roughly twelve to fifteen hours per week. In an eight-week course, missing one week is roughly equivalent to missing two in a regular semester.

3.2 Submissions

Grades are weighted by group: weekly technical assignments 40%, research paper presentation 10%, course project 40%, course project presentation 10%.

- **Weekly technical assignments** are submitted through the module assignment. Accepted formats are specified in each assignment and may include a short PDF write-up, a notebook, a zipped code folder, or a link to a public repository with run instructions. Interpret your results; do not just paste output.
- **Project track:** every project deliverable in Modules 1 through 8 is a graded discussion. Reply to the discussion post and attach PDFs, notebooks, or repository links. Formal milestones are the proposal (Module 2), mid-project report (Module 5), draft report (Module 7), and final report plus recorded presentation (Module 8); the other weeks are informal check-ins.
- **Research paper presentation** (Module 7) and **final project presentation** (Module 8) are recorded videos of twelve to fifteen minutes, uploaded or linked, with the slide deck attached separately and captions or a transcript provided.
- **Formats:** PDF preferred for written work; notebook or zipped folder or public repository for code.
- **AI tools** are encouraged for learning, debugging, and drafting; you remain responsible for the correctness, originality, and integrity of everything you submit, and for being able to explain it in your own words. ODU academic-integrity policies apply in full.

3.3 Communication

Use the course inbox for individual questions (response target: one business day on weekdays) and ODU email for personal or sensitive matters. Weekly student hours are held by Zoom at a time posted in the announcements. Post class-wide questions (assignment clarifications, common tool problems, conceptual questions) to the Q&A Café discussion forum so that the answer helps everyone. Check the course site at least twice a week; announcements are posted at least weekly.

3.4 Project expectations

You will design, develop, document, and present a generative-AI project applied to a real problem, dataset, or research question. The topic must relate to generative AI and/or agentic AI. Work individually or in an instructor-approved small team (approval happens before the proposal is finalized). Build incrementally across the whole term; do not back-load the work into the final two weeks. Deliverables are a written proposal (problem, methods, data, references), a mid-project report, a draft or progress report, a final written report with code or a reproducible repository, and a recorded final presentation. Evaluation weighs scope, technical soundness, evidence of progress, appropriate methods, reproducibility, interpretation of results, and quality of written and spoken communication. Ideally at least one of the three paper candidates you propose in Module 1 for your paper presentation should relate to one of your project topics: reading deeply for the presentation directly strengthens the project.

4 Topic 4: Introductory Computing Setup for Local or Cloud Execution

Most coursework (training small generative models, running notebooks, fine-tuning transformers, agent experiments) is far easier with a GPU. Do not skip this in Week 1: account approval can take several business days, and the Module 1 setup discussion asks you to confirm HPC access.

4.1 Three tiers of compute

1. **Your laptop or desktop (CPU only).** Fine for reading code, debugging on tiny inputs, and small VAE experiments. Not realistic for transformers, diffusion, or multimodal work beyond toy scale.
2. **ODU HPC (recommended).** Free for ODU students, real GPUs, professionally administered. Two clusters serve this course: **Wahab** (default) and **Waterfield** (fallback, ODU's newer Google-Cloud cluster; use it when Wahab is congested or a job needs a newer GPU type). Both use the same MIDAS credentials and the same one-time access request.
3. **Google Colab.** Zero-install notebook with a free GPU tier; good for prototypes and quick exploration. Free-tier sessions disconnect and long runs get killed, so treat Colab as supplemental rather than primary.

4.2 Step-by-step setup

1. **Request an HPC account on Day 1.** Submit the ODU Research and Cloud Computing (RCC) access request form at `https://forms.odu.edu/view.php?id=93440`. One form covers Wahab, Turing, and Waterfield. You will be emailed when the account is provisioned.
2. **Pick how to connect.** New HPC users should start with the browser portal (Open OnDemand), which hides SLURM and the Linux shell and launches a GPU JupyterLab session in a few clicks: Wahab at `https://ondemand.wahab.hpc.odu.edu`, Waterfield at `https://ondemand.waterfield.hpc.odu.edu`. SSH is for those already comfortable with a terminal. Waterfield accepts `ssh <midas_username>@waterfield.hpc.odu.edu`. For Wahab, confirm the current SSH login hostname with RCC (`rcc@odu.edu`, Tuesday office hours) or the HPC wiki before configuring keys; it is deliberately not listed here. All routes require MIDAS credentials and Duo two-factor approval.

3. **Launch JupyterLab with a GPU.** In Open OnDemand, open *Interactive Apps*, choose *JupyterLab*, pick a GPU partition, set a walltime of one to two hours, launch, then *Connect to JupyterLab*. If the partition is busy, retry with a shorter walltime or switch to Waterfield.
4. **Shell users:** software is managed with LMOD modules (`module avail`, `module load python3`); on Waterfield applications run through a container wrapper (`crun`, for example `module load pytorch-gpu` then `crun.pytorch-gpu python train.py`). Use a Python virtual environment for project-specific dependencies rather than requesting a system-wide install. See https://wiki.hpc.odu.edu/en/Software/Python.
5. **Storage:** `/home/<midas>` is persistent (code, configuration, small data; back up anything irreplaceable); `/scratch/<midas>` is fast but not backed up (temporary job space; clean it up); `/cm/shared` holds shared read-only software. Move data between clusters with `scp` or `rsync` over SSH.

4.3 Mapping compute to the modules

Modules 2 (VAE) and 4 (autoregressive) fit on a laptop CPU or Colab, though Wahab is more comfortable. Modules 3, 5, 6, 7, and 8 should run on a real GPU: Wahab via Open OnDemand by default, Waterfield if congested or if newer GPUs are needed. Scope your project to the GPU budget you actually have; if you intend to train a large transformer or diffusion model from scratch, plan early for Waterfield rather than Colab.

Help and readings. RCC offers Tuesday office hours and `rcc@odu.edu`. Before your first job, read the RCC "Introduction to HPC" workshop page (https://wiki.hpc.odu.edu/Training/HPC-Intro) and the Open OnDemand walkthrough (https://wiki.hpc.odu.edu/en/open-ondemand). New to the UNIX shell? Software Carpentry's lesson at http://swcarpentry.github.io/shell-novice/ takes about two hours.